CobaltStrike Beacon使用知名的开源静态链接库：libtomcrypt

在逆向Beacon时，我发现Beacon使用了 `rng_get_bytes`

libtomcrypt / src / prngs / rng_get_bytes.c

Open symbols panel

Code Blame 167 lines (142 loc) · 4.01 KB Raw

```c
    static HCRYPTPROV hProv = 0;
    LTC_UNUSED_PARAM(callback);

    if (hProv == 0) {
       HCRYPTPROV h = 0;
       if (!CryptAcquireContextW(&h, NULL, MS_DEF_PROV_W, PROV_RSA_FULL,
                                 (CRYPT_VERIFYCONTEXT | CRYPT_MACHINE_KEYSET)) &&
           !CryptAcquireContextW(&h, NULL, MS_DEF_PROV_W, PROV_RSA_FULL,
                                 CRYPT_VERIFYCONTEXT | CRYPT_MACHINE_KEYSET | CRYPT_NEWKEYSET)) {
          return 0;
       }
       hProv = h;
    }

    return CryptGenRandom(hProv, (DWORD)len, (BYTE *)buf) == TRUE ? len : 0;
 }
#endif /* Old WIN32 versions */
#endif /* WIN32 */

/**
  Read the system RNG
```

这可能成为超越Wininet所属Api的更为有力的一种检测方法，根据CobaltStrike官方发布的4.10版本信息可以观察到，Fortra推出了一个名为Beacon_Gate的规避方案供用户选择，beacon_gate.h

Files

855a33a

Go to file

- BOF-Template
  - base
  - BOF-Template.vcxproj
  - BOF-Template.vcxproj.filters
  - Makefile
  - beacon.h
  - beacon_gate.h
  - bof.cpp
  - packages.config
  - sleepmask.h
- .gitignore
- BOF-Template.sln
- LICENSE
- README.md

bof-vs / BOF-Template / beacon_gate.h

Top

Code Blame 69 lines (65 loc) · 2.69 KB Raw

```c
 *
 * functionPtr - target function to call
 * function - Enum representing target WinApi
 * numOfArgs - number of arguments
 * args - array of ULONG_PTRs containing the passed arguments (e.g. rcx, rdx, ...)
 * bMask - BOOL indicating whether Beacon should be masked during the call
 * ULONG_PTR - retValue of the atomic function call
 */
typedef struct {
    PVOID functionPtr;
    WinApi function;
    int numOfArgs;
    ULONG_PTR args[MAX_BEACON_GATE_ARGUMENTS];
    BOOL bMask;
    ULONG_PTR retValue;
} FUNCTION_CALL, * PFUNCTION_CALL;

/* Currently support max 10 arguments. */
/* NB For x86 we only support std call convention as this is what Windows uses for most Win32 APIs. */
typedef ULONG_PTR(__stdcall* BEACON_GATE_00)(VOID);
typedef ULONG_PTR(__stdcall* BEACON_GATE_01)(ULONG_PTR);
typedef ULONG_PTR(__stdcall* BEACON_GATE_02)(ULONG_PTR, ULONG_PTR);
typedef ULONG_PTR(__stdcall* BEACON_GATE_03)(ULONG_PTR, ULONG_PTR, ULONG_PTR);
typedef ULONG_PTR(__stdcall* BEACON_GATE_04)(ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR);
typedef ULONG_PTR(__stdcall* BEACON_GATE_05)(ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR);
typedef ULONG_PTR(__stdcall* BEACON_GATE_06)(ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR);
typedef ULONG_PTR(__stdcall* BEACON_GATE_07)(ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR);
typedef ULONG_PTR(__stdcall* BEACON_GATE_08)(ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR);
typedef ULONG_PTR(__stdcall* BEACON_GATE_09)(ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR);
typedef ULONG_PTR(__stdcall* BEACON_GATE_10)(ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR, ULONG_PTR);
#endif // _BEACON_GATE_H
```

可以简单的推测，BeaconGate的大概样子。

我通过自己编写一些yara规则在互联网上获取到了一些样本。这很有趣~

我尝试验证我的观点，我们去给这几个有意思的函数下断点：CryptAcquireContext、CryptGenRandom、CryptReleaseContext

因为这些函数都是转发调用的，所以我们需要断点stub

```
00007ffe`a5f13dc0 cc              int     3
0:011> bp advapi32!CryptAcquireContextAStub
0:011> g
Breakpoint 0 hit
ADVAPI32!CryptAcquireContextAStub:
00007ffe`a3d6b6b0 48ff25592a0a00  jmp     qword ptr [ADVAPI32!_imp_CryptAcquireContextA (00007ffe`a3e0e110)] ds:00007ffe`a3e0e110={CRYPTSP!CryptAcquireContex
0:002> knf
 #   Memory  Child-SP          RetAddr               Call Site
00           00000000`00dbfce8 00000000`00dc01be     ADVAPI32!CryptAcquireContextAStub
01         8 00000000`00dbfcf0 00000000`00000000     0xdc01be
0:002>
```

Locals

Name	Value	Ty

Stack

Frame Index	Name
[0x0]	**ADVAPI32!CryptAcquireContextAStub**
[0x1]	0xdc01be

```
Breakpoint 0 hit
ADVAPI32!CryptAcquireContextAStub:
00007ffe`a3d6b6b0 48ff25592a0a00  jmp     qword ptr [ADVAPI32!_imp_CryptAcquireContextA (00007ffe`a3e0e110)] ds:00007ffe`a3e0e110={CRYPTSP!CryptAcquireContex
0:002> knf
 #   Memory  Child-SP          RetAddr               Call Site
00           00000000`00dbfce8 00000000`00dc01be     ADVAPI32!CryptAcquireContextAStub
01         8 00000000`00dbfcf0 00000000`00000000     0xdc01be
0:002> bp ADVAPI32!CryptReleaseContextStub
0:002> g
Breakpoint 1 hit
ADVAPI32!CryptReleaseContextStub:
00007ffe`a3d6b890 48ff25a1280a00  jmp     qword ptr [ADVAPI32!_imp_CryptReleaseContext (00007ffe`a3e0e138)] ds:00007ffe`a3e0e138={CRYPTSP!CryptReleaseContext
0:002> knf
 #   Memory  Child-SP          RetAddr               Call Site
00           00000000`00dbfce8 00000000`00dc020a     ADVAPI32!CryptReleaseContextStub
01         8 00000000`00dbfcf0 00000000`00000004     0xdc020a
02         8 00000000`00dbfcf8 00000000`00000000     0x4
0:002>
```

Locals

Name	Value	Ty

Stack

Frame Index	Name
[0x0]	**ADVAPI32!CryptReleaseContextStub**
[0x1]	0xdc020a
[0x2]	0x4

```
0:002> bp ADVAPI32!CryptGenRandomStub
0:002> g
Breakpoint 0 hit
ADVAPI32!CryptAcquireContextAStub:
00007ffe`a3d6b6b0 48ff25592a0a00  jmp     qword ptr [ADVAPI32!_imp_CryptAcquireContextA (00007ffe`a3e0e110)] ds:00007ffe`a3e0e110={CRYPTSP!CryptAcquireContex
0:002> knf
 #   Memory  Child-SP          RetAddr               Call Site
00           00000000`00dbfce8 00000000`00dc01be     ADVAPI32!CryptAcquireContextAStub
01         8 00000000`00dbfcf0 00000000`00000000     0xdc01be
0:002>
```

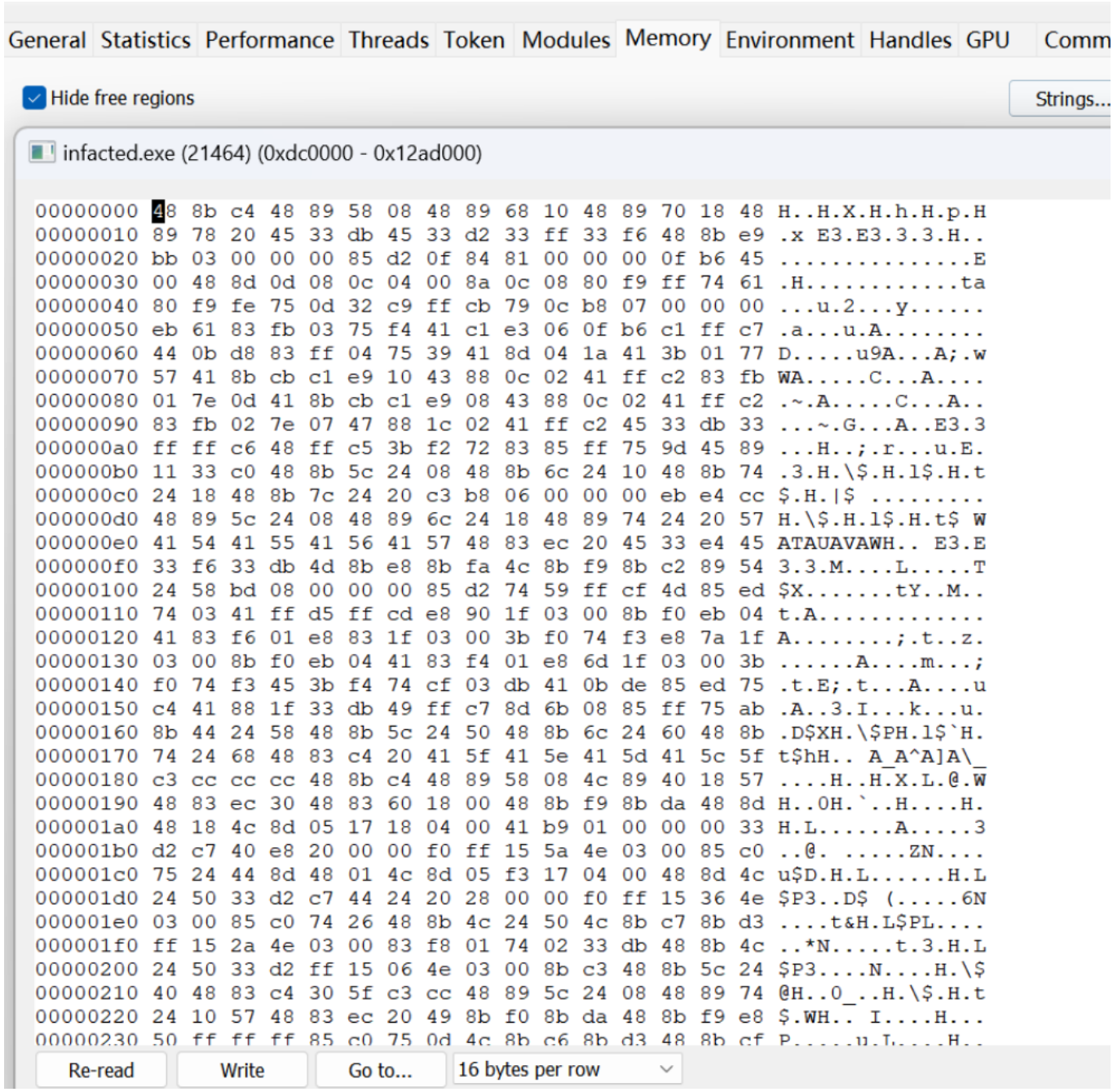
Locals

Name	Value	Ty

Stack

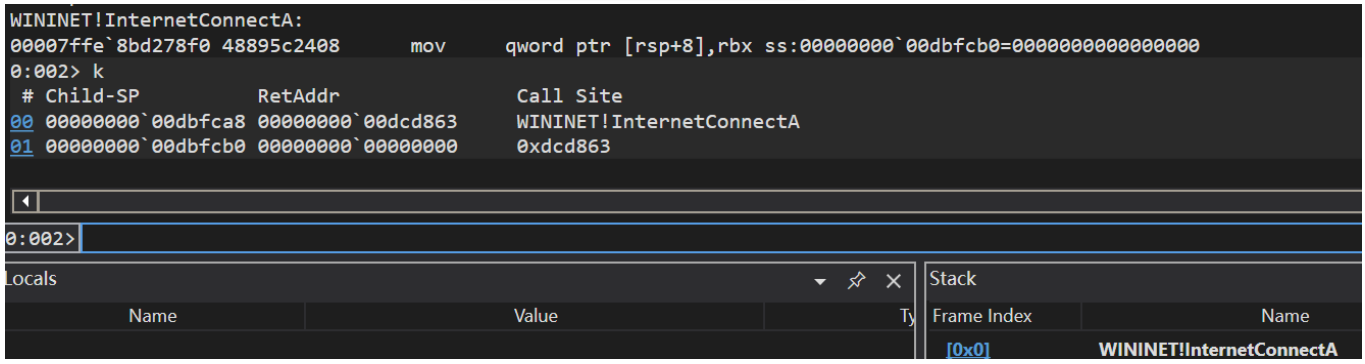
Frame Index	Name
[0x0]	**ADVAPI32!CryptAcquireContextAStub**
[0x1]	0xdc01be

Oops看我们找到了什么，一个奇怪的调用者（这很奇怪不是么）



Aha~这很CobaltStrike不是么。

我们在尝试一下他们所提到了的 `WININET!InternetOpenA`



很显然也被命中了，当然他们在Blog中提到已经对这个Api进行了处理，但是并没有CryptAcquireContext、CryptGenRandom、CryptReleaseContext这些，并且这出现误报的几率

非常小（个人拙见）

在探索其他样本的过程中，我觉得我可能捕获了4.10版本的Beacon，请看这个奇怪的地址

```
0:000> k
 # Child-SP          RetAddr           Call Site
00 000000c0`dbeff898 000001f0`19cc09e0 WININET!InternetConnectA
01 000000c0`dbeff8a0 00000000`00000000 0x000001f0`19cc09e0
0:000> g
Breakpoint 1 hit
WININET!InternetConnectA:
00007ffe`8bd278f0 48895c2408      mov     qword ptr [rsp+8],rbx ss:000000c0`dbeff8a0=0000000000000000
0:000> k
 # Child-SP          RetAddr           Call Site
00 000000c0`dbeff898 000001f0`19cc09e0 WININET!InternetConnectA
01 000000c0`dbeff8a0 00000000`00000000 0x000001f0`19cc09e0
```

Base address	Type	Size	Protection	Use
0x1f0186f0000	Mapped	1,540 kB	R	
0x1f018880000	Mapped	20,484 kB	R	
0x1f019c90000	Private	4 kB	RW	
0x1f019ca0000	Mapped	68 kB	R	C:\Windows\System32\C_1252.NLS
0x1f019cc0000	Private	12 kB	RW	
0x1f019cc0000	Private: Commit	8 kB	RX	
0x1f019cc2000	Private: Commit	4 kB	RW	

60688) (0x1f019cc0000 - 0x1f019cc2000)

```
89 4c 24 08 48 83 ec 28 48 8b 44 24 30 48 83 H.L$.H..(H.D$0H.
20 00 75 05 e9 cf 00 00 00 c7 44 24 04 00 00 x .u.......D$...
00 8b 44 24 04 48 6b c0 10 48 8b 4c 24 30 48 ...D$.Hk..H.L$0H
49 20 48 83 3c 01 00 0f 84 ab 00 00 00 c7 04 .I H.<..........
00 00 00 00 eb 08 8b 04 24 ff c0 89 04 24 8b $........$....$.
24 8b 4c 24 04 48 6b c9 10 48 8b 54 24 30 48 .$.L$.Hk..H.T$0H
52 20 48 3b 44 0a 08 73 70 8b 44 24 04 48 6b .R H;D..sp.D$.Hk
10 48 8b 4c 24 30 48 8b 49 20 8b 14 24 48 89 ..H.L$0H.I ..$H.
24 10 48 8b 04 01 48 89 44 24 08 33 d2 8b 04 T$.H...H.D$.3...
b9 0d 00 00 00 f7 f1 8b c2 8b c0 48 8b 4c 24 $...........H.L$
0f be 44 01 28 48 8b 4c 24 08 48 8b 54 24 10 0..D.(H.L$.H.T$.
be 0c 11 33 c8 8b c1 8b 4c 24 04 48 6b c9 10 ....3....L$.Hk..
8b 54 24 30 48 8b 52 20 44 8b 04 24 48 8b 0c H.T$0H.R D..$H..
42 88 04 01 e9 6d ff ff ff 8b 44 24 04 ff c0 .B....m....D$...
```

这应该是SleepMask，正如Fortra所说到的

In Cobalt Strike 4.10, we have taken this idea to its logical conclusion and the Sleepmask now supports the execution of arbitrary functions. Therefore, it is now possible to configure Beacon to forward its Windows API calls to be executed via the Sleepmask (aka BeaconGate).

这激发了我强烈的兴趣，虽然我是一个红队菜鸟，我尝试提取出内存中Beacon，并且逆向它，

```
BOOL __stdcall DllMain(HINSTANCE hinstDLL, DWORD fdwReason, LPVOID lpvReserved)
{
  DWORD v5; // edx
  DWORD v6; // edx
  struct _MEMORY_BASIC_INFORMATION mbi; // [rsp+20h] [rbp-38h] BYREF

  v5 = fdwReason - 1;
  if ( v5 )
  {
    v6 = v5 - 3;
    if ( v6 )
    {
      if ( v6 == 9 )
        Beacon_init(lpvReserved);
    }
    else
    {
      if ( Beacon_short(38i64) == 1 && hinstDLL && BeaconVirtualQuery(hinstDLL, &mbi, 0x30ui64) )
      {
        if ( mbi.Type == 0x20000 )
        {
          if ( *(mbi.AllocationBase + 4) == 0xFFEEFFEE )
          {
            if ( *(mbi.AllocationBase + 6) == mbi.AllocationBase )
              HeapDestroy(*(mbi.AllocationBase + 5));
          }
          else if ( !BeaconVirtualFree(hinstDLL, 0i64, 0x8000u) )
          {
            BeaconVirtualFree(mbi.AllocationBase, 0i64, 0x8000u);
          }
        }
        else if ( mbi.Type == 0x40000 )
        {
          BeaconUnmapViewOfFile(hinstDLL);
        }
      }
      beacon(lpvReserved);
    }
  }
```

这和之前的版本都有很大的差异，我发现VirtualQuery等Api都经过了缜密的自定义处理，对应的这大概就是Fortra所说的新增加Api

LPVOID **BeaconVirtualAllocEx**(HANDLE processHandle, LPVOID lpAddress, SIZE_T dwSize, DWORD flAllocationType, DWORD flProtect);

BOOL **BeaconVirtualProtect**(LPVOID lpAddress, SIZE_T dwSize, DWORD flNewProtect, PDWORD lpflOldProtect);

BOOL **BeaconVirtualProtectEx**(HANDLE processHandle, LPVOID lpAddress, SIZE_T dwSize, DWORD flNewProtect, PDWORD lpflOldProtect);

BOOL **BeaconVirtualFree**(LPVOID lpAddress, SIZE_T dwSize, DWORD dwFreeType);

BOOL **BeaconGetThreadContext**(HANDLE threadHandle, PCONTEXT threadContext);

BOOL **BeaconSetThreadContext**(HANDLE threadHandle, PCONTEXT threadContext);

DWORD **BeaconResumeThread**(HANDLE threadHandle);

HANDLE **BeaconOpenProcess**(DWORD desiredAccess, BOOL inheritHandle, DWORD processId);

HANDLE **BeaconOpenThread**(DWORD desiredAccess, BOOL inheritHandle, DWORD threadId);

BOOL **BeaconCloseHandle**(HANDLE object);

BOOL **BeaconUnmapViewOfFile**(LPCVOID baseAddress);

SIZE_T **BeaconVirtualQuery**(LPCVOID address, PMEMORY_BASIC_INFORMATION buffer, SIZE_T length);

BOOL **BeaconDuplicateHandle**(HANDLE hSourceProcessHandle, HANDLE hSourceHandle, HANDLE hTargetProcessHandle, LPHANDLE lpTargetHandle, DWORD dwDesiredAccess, BOOL bInheritHandle, DWORD dwOptions);

```
__int64 __fastcall BeaconVirtualQuery(_BYTE *address, PMEMORY_BASIC_INFORMATION buffer, _BYTE *length)
{
  _DWORD *result1; // rax
  __int64 (__fastcall *result3)(HANDLE, _BYTE *, _QWORD, PMEMORY_BASIC_INFORMATION, _BYTE *, __int64 *); // rdi
  HANDLE v9; // rax
  int queryFunction; // eax
  HANDLE currentProcess; // rax
  __int64 errorCode; // [rsp+58h] [rbp+20h] BYREF

  if ( sub_1800144EC(18) )
  {
    createDataStructure(0, length, 0i64, 0);
    createDataStructure(2, buffer, 48i64, 1);
    result1 = createDataStructure(0, address, 0i64, 0);
    return sub_18001459C(VirtualQuery, 18, 3, result1);
  }
  else if ( dword_180056270
         && ((result3 = qword_18005A6F8, dword_18005626C == 1) && qword_18005A6F8
          || dword_18005626C == 2 && qword_18005A700)
         && ((errorCode = 0i64, dword_18005626C != 1)
           ? (qword_180056260 = qword_18005A700,
              dword_180056268 = dword_18005A708,
              currentProcess = GetCurrentProcess(),
              queryFunction = sub_180023667(currentProcess, address, 0i64, buffer, length, &errorCode))
           : (v9 = GetCurrentProcess(), queryFunction = result3(v9, address, 0i64, buffer, length, &errorCode)),
             !queryFunction) )
  {
    return errorCode;
  }
  else
  {
    return VirtualQuery(address, buffer, length);
  }
}
```

通过对其深入探究我发现了许多有意思的事情，当然这里不做讨论，且听下回分解。言归正传，我们现在就可以新增加关于CryptAcquireContextA、CryptGenRandom、CryptReleaseContext的调用检查来抓住内存中的Beacon。

关于检测的Demo，我们可以修改一个开源的优秀项目，https://github.com/waldo-irc/MalMemDetect/tree/main

进行一些调整，POC如下：

```
// Hooked CryptAcquireContextA with return address and module checking
HCRYPTPROV WINAPI HookedCryptAcquireContextA(HCRYPTPROV *phProv, LPCSTR
pszContainer, LPCSTR pszProvider, DWORD dwProvType, DWORD dwFlags) {
    DWORD callerId = GetCurrentThreadId();
    CHAR* log = (CHAR*)malloc(256);
    HMODULE hModule;
    char lpBaseName[256] = { 0 };

    // Check if the return address belongs to known libraries
    if (memcmp(_ReturnAddress(), "\x4c\x8b\xc0\x48\x85\xc0\x0f\x84", 8) == 0
|| memcmp(_ReturnAddress(), "\x48\x8b\xd8\x48\x85\xc0\x0f\x84", 8) == 0) {
        strcpy_s(lpBaseName, "ntdll.dll");
    } else {
        if (::GetModuleHandleExA(GET_MODULE_HANDLE_EX_FLAG_FROM_ADDRESS,
(LPCSTR)_ReturnAddress(), &hModule) == 1) {
            ::GetModuleBaseNameA(GetCurrentProcess(), hModule, lpBaseName,
sizeof(lpBaseName));
        }
```

```
        snprintf(log, 255, "Suspicious CryptAcquireContextA() call from thread
id:%d, pszContainer: %s, pszProvider: %s\n", callerId, pszContainer ?
pszContainer : "NULL", pszProvider ? pszProvider : "NULL");
        LogDetected(&log);
    }

    std::string modName = lpBaseName;
    std::transform(modName.begin(), modName.end(), modName.begin(),
        [](unsigned char c) { return tolower(c); });

    if (dllMonitor != "NULL" && dllMonitor.find(modName) != std::string::npos)
{
        snprintf(log, 255, "Suspicious CryptAcquireContextA() call from
module: %s, pszContainer: %s\n", dllMonitor.c_str(), pszContainer ?
pszContainer : "NULL");
        LogDetected(&log);
    }

    free(log);
    return OldCryptAcquireContextA(phProv, pszContainer, pszProvider,
dwProvType, dwFlags);
}

// Hooked CryptGenRandom with return address and module checking
BOOL WINAPI HookedCryptGenRandom(HCRYPTPROV hProv, DWORD dwLen, BYTE
*pbBuffer) {
    DWORD callerId = GetCurrentThreadId();
    CHAR* log = (CHAR*)malloc(256);
    HMODULE hModule;
    char lpBaseName[256] = { 0 };

    if (memcmp(_ReturnAddress(), "\x4c\x8b\xc0\x48\x85\xc0\x0f\x84", 8) == 0
|| memcmp(_ReturnAddress(), "\x48\x8b\xd8\x48\x85\xc0\x0f\x84", 8) == 0) {
        strcpy_s(lpBaseName, "ntdll.dll");
    } else {
        if (::GetModuleHandleExA(GET_MODULE_HANDLE_EX_FLAG_FROM_ADDRESS,
(LPCSTR)_ReturnAddress(), &hModule) == 1) {
            ::GetModuleBaseNameA(GetCurrentProcess(), hModule, lpBaseName,
sizeof(lpBaseName));
        }
        snprintf(log, 255, "Suspicious CryptGenRandom() call from thread
id:%d, Length: %lu\n", callerId, dwLen);
        LogDetected(&log);
    }

    std::string modName = lpBaseName;
```

```
    std::transform(modName.begin(), modName.end(), modName.begin(),
        [](unsigned char c) { return tolower(c); });

    if (dllMonitor != "NULL" && dllMonitor.find(modName) != std::string::npos)
{
        snprintf(log, 255, "Suspicious CryptGenRandom() call from module: %s,
Length: %lu\n", dllMonitor.c_str(), dwLen);
        LogDetected(&log);
    }

    free(log);
    return OldCryptGenRandom(hProv, dwLen, pbBuffer);
}

// Hooked CryptReleaseContext with return address and module checking
BOOL WINAPI HookedCryptReleaseContext(HCRYPTPROV hProv, DWORD dwFlags) {
    DWORD callerId = GetCurrentThreadId();
    CHAR* log = (CHAR*)malloc(256);
    HMODULE hModule;
    char lpBaseName[256] = { 0 };

    if (memcmp(_ReturnAddress(), "\x4c\x8b\xc0\x48\x85\xc0\x0f\x84", 8) == 0
|| memcmp(_ReturnAddress(), "\x48\x8b\xd8\x48\x85\xc0\x0f\x84", 8) == 0) {
        strcpy_s(lpBaseName, "ntdll.dll");
    } else {
        if (::GetModuleHandleExA(GET_MODULE_HANDLE_EX_FLAG_FROM_ADDRESS,
(LPCSTR)_ReturnAddress(), &hModule) == 1) {
            ::GetModuleBaseNameA(GetCurrentProcess(), hModule, lpBaseName,
sizeof(lpBaseName));
        }
        snprintf(log, 255, "Suspicious CryptReleaseContext() call from thread
id:%d, Flags: %lu\n", callerId, dwFlags);
        LogDetected(&log);
    }

    std::string modName = lpBaseName;
    std::transform(modName.begin(), modName.end(), modName.begin(),
        [](unsigned char c) { return tolower(c); });

    if (dllMonitor != "NULL" && dllMonitor.find(modName) != std::string::npos)
{
        snprintf(log, 255, "Suspicious CryptReleaseContext() call from module:
%s, Flags: %lu\n", dllMonitor.c_str(), dwFlags);
        LogDetected(&log);
    }
```

```
    free(log);
    return OldCryptReleaseContext(hProv, dwFlags);
}
```